

Intrusion detection in networks using ensemble-based deep federated learning

Dhanraj Somaling Jadhav^{a,*}, Sneha R Jadhav-Mane^b, Kayan Devappa Bamane^c,
Sonal Suresh Mohite^c, Abhijit Janardan Patankar^a, Arati Sidu Gaikwad^a

^a Department of Information Technology, DY Patil College of Engineering, Akurdi, Pune-411044, India

^b Department of Information Technology, Brahmdevdada Mane Institute of Technology, Solapur, 413002, India

^c Department of Computer Engineering, D Y Patil College of Engineering, Akurdi, Pune-411044, India

ARTICLE INFO

Keywords:

Internet of Things (IoT)
Intrusion detection system (IDS)
Federated learning (FL)
Ensemble-based deep federated learning model (E-DFL)
Residual auto-encoder
Attention capsule network
Stacked Bi-LSTM
Edge-IIoTset dataset

ABSTRACT

The enormous development of Internet of Things (IoT) devices poses unique challenges for interconnected systems in relation to privacy and security. The increasing number of cyber-attacks has made scalable intrusion detection systems (IDS) mandatory for all organizations. When dealing with cyber-attacks, existing methodologies have struggled to capture spatial and temporal features. When the number of edge systems rises, tradeoffs exist due to a lack of diversity. Thus, Federated Learning (FL) based IDS is proposed to ensure privacy preservation, accurate detection of attacks, and cost reduction, which are difficult in old centralized machine-learning-based IDS. The FL-based model offers privacy benefits but still faces vulnerabilities that are evaded by adversaries through the cooperation of user data. Therefore, ensemble models were evaluated as possible solutions for improving attack detection and classification. As a result, the proposed study introduced an ensemble-based deep federated learning model (E-DFL) driven IDS framework for IoT applications. The concept uses a residual auto-encoder, attention capsule network, and stacked bidirectional long short-term memory (Bi-LSTM) as basis classifiers to generate ensemble data. The meta-classifier then performs the final intrusion detection on the test set using a global model created by combining ensemble model updates from different clients. Compared to classical FL approaches, the E-DFL model provides improved user privacy. Edge-IIoT cybersecurity datasets were used to assess the effectiveness of the proposed E-DFL method, and achieved an accuracy of 99.23%.

1. Introduction

Information and communication technology (ICT) incorporates a wide range of technologies that lead to the creation, transmission and reception of information [1]. The ICT allows people to communicate, work and interact with each other worldwide. The widespread use of ICT and its advancements have reshaped daily activities. ICT improves decision-making for individuals and organizations by enabling real-time operations [2,3]. The transfer of digital information across the network introduces vulnerabilities that harm both businesses and individuals. Therefore, strong network safety measures are essential to protect privacy, veracity, and accessibility. Network security is regarded as the first line in multilayer network design [4,5]. An IDS continuously monitors the network for suspicious behaviour and sends notifications when threats are detected. Early detection prevents data breaches and allows for remediation. The IDS employs hardware or software to monitor

intrusions in several locations [6].

The IoT is a type of communication technology that employs sensors to connect items to the internet, allowing them to perceive objects, talk with one another, as well as transmit data [7]. The total IoT devices linked to the network is growing by the day; some applications include agriculture, e-health, and so on. Some of the problems connected with IoT device adoption include security, scalability, and reliability. IoT devices are linked to the internet via susceptible communication protocols, which causes security vulnerabilities [8,9]. To mitigate security threats, these issues provide challenges to the IoT ecosystem and require collaboration among diverse standards and protocols. The distributed nature of IoT applications raises network organization as well as security concerns. Therefore, IDS is needed to protect IoT applications from cyber threats [10]. This problem can be solved using either reactive or proactive techniques.

The IDS is divided into 3 types: signature-based, anomaly-based, and

* Corresponding author.

E-mail address: dsjadhav0831@gmail.com (D.S. Jadhav).

<https://doi.org/10.1016/j.knosys.2026.115491>

Received 10 December 2024; Received in revised form 12 January 2026; Accepted 3 February 2026

Available online 5 February 2026

0950-7051/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

hybrid IDS. The signature-based method is inefficient since it is only applicable for known attacks and requires specialist knowledge to detect threats [11]. The anomaly-based IDS gained popularity since it requires less human interaction, whereas the hybrid approach combines signature as well as anomaly-based IDS. Since the application of signature-based methods is limited in IoT networks, anomaly-based IDS plays an essential role in IoT network IDS [12,13]. Common network attacks include denial of service (DoS), user-to-root (U2R), probes, and remote-to-local (R2L). However, due to the large number of IoT devices connected to the network, these assaults are difficult to detect. The existing studies used machine learning techniques to detect intrusions [14]. However, a substantial amount of data produced by IoT devices presents obstacles for traditional data processing methods; thus, ML methods are regarded as helpful for extensive research. Thus, research used deep learning (DL) methods to detect intrusions, which extracted information more reliably than traditional learning algorithms [15,16].

The DL methods contain a convolutional neural network (CNN), a deep belief network (DBN), a recurrent neural network (RNN), etc. DL models use a layer hierarchy to extract features from input and train in a supervised, unsupervised, or semi-supervised manner. Different networks experience varied attacks due to specific configurations, network topology, etc. Hence, it is necessary to collect data from multiple networks and train the model to detect attacks [17,18]. However, transmitting data to the centralized server is difficult due to high costs, computational overhead, increased complexity, and privacy issues. Due to privacy issues, storing data on a centralized server is feasible. In IoT networks, every node has the ability to process data locally; hence, instead of training data at a centralized server, each node is designed to train the model locally and only share parameters with the central server [19,20]. The server aggregates the parameters to form an aggregated model that is then used to detect intrusions.

Motivation: IoT technologies play an important part in the transformation of systems, including smart farming, smart cities, and smart health. However, large-scale deployment of IoT devices raises security concerns due to their placement in an unsupervised environment. Because IoT devices are linked via wireless networks, hackers can tap into the communication channel and obtain access to sensitive data. IoT devices often face challenges in implementing robust security measures due to limited energy and computational resources. This necessitates the development of preventative measures to protect against threats. IDS have been introduced to protect IoT systems from threats; however, due to limitations in capacity, energy, connectivity, etc., traditional IDS are less effective and insufficient. As a result, ML/DL approaches are utilized to detect both benign and abnormal behaviour. Thus, the suggested study introduced an E-DFL for detecting network intrusions. The main goals of the suggested intrusion detection system are outlined below:

- To confirm enhanced security against unwanted attacks, an E-DFL approach for IoT intrusion detection was proposed. The DFL enhances privacy and effectively detects altered types of attacks on IoT networks.
- To improve intrusion detection, three deep learning classifiers (residual AE, stacked Bi-LSTM, and attention capsule net) are combined to generate weights for the local model. These weights are then used to extract spatial and temporal information to detect various types of attacks.
- To aggregate the different user weights, the federated server utilized federated averaging with momentum to generate aggregated weights.
- To assess the performance against the traditional centralized learning method utilizing the Edge-IIoTset dataset.

This study is structured as follows: Section 2 discusses current research on the Federated learning-based DL approach for intrusion recognition in IoT systems. The proposed ensemble-based deep federated learning is described in Section 3 of the paper. Results as well as

discussion are explained in Section 4. Section 5 discusses the conclusion as well as future work.

2. Related work

The development of communication technology led to a rise in the number of connected systems and network traffic. Thus, Ons Aouedi et al. [21] adopted a federated blending method for intrusion detection (F-BIDS). The FL model faced vulnerabilities; thus, blending models is used to enhance attack detection by combining multiple models. The data collected from the Edge-IIoTset and InSDN datasets were pre-processed using feature mapping to transform categorical features, followed by min-max normalization. Then, feature selection is performed using recursive feature elimination, where the selected features are fed to a federated blending model to identify intrusions in the network. The blending emulate creates a new meta-dataset with random forest (RF) and decision tree (DT) and uploads model parameters to the federated server. The local model parameters are combined via federated averaging to create a new global model. The aggregated global model parameters are applied to local models, resulting in threat detection utilizing the global model.

Othmane Friha et al. [22] presented an FL-based intrusion detection system (FELIDS) for agricultural IoT networks. FELIDS safeguards data privacy by local learning, exchanging only model modifications with the federated server. To detect IoT assaults, FELIDS employed three classifiers: DNN, CNN, as well as RNN. The performance is measured using 3 datasets, such as CSE-CIC-IDS2018, MQTTset, as well as InSDN. The study found that FELIDS outperformed non-FL-based models in terms of privacy protection and threat identification.

IoT connects more devices to the internet through a wireless connection. The interconnected nature of IoT devices often leads to a significant transfer of data between users, which can create vulnerabilities and privacy concerns. As a result, Md Mamunur Rashid et al. [23] implemented an FL-based intrusion detection algorithm for IoT systems that employed a decentralized ML model. The FL system ensured the confidentiality and privacy of local IoT data. The study used the Edge-IIoTset dataset to analyze the performance and attained an accuracy of 92.49 %. It has high latency, an overloaded network, and [privacy leakage](#).

Yanu et al. [24] demonstrated IDS using FL and edge computing to provide cybersecurity to IoT applications. The FL system aggregated data from multiple users to train the model, resulting in a security risk within the FL framework. In the FL process, each user has access to the method parameters and data to be trained; therefore, malicious individuals may change the weights and data. Thus, a detection method in IDS was implemented to identify poisoning assaults, in which a centralized server filters abnormalities and removes their parameters from the global aggregation process. In addition, a scoring mechanism is used to evaluate harmful users based on the loss of local model data. The similarity was then calculated using scores to identify anomalous users. The performances were analyzed using the CIC-IDS-2017 dataset, and the accuracy was 97.1 %.

Cyber-attacks in the IoT environment have increased the use of interconnected systems and computer networks. The usage of ML-based intrusion detection systems provided prominent results but still faces privacy and regulatory issues. As a result, federated learning systems emerged as a viable option that preserved local data privacy by using a larger number of clients to train the local model. Idrissi et al. [25] adopted Fed-ANIDS, which used anomaly detection and federated learning. The local models under consideration are auto-encoders, variational auto-encoders, as well as adversarial auto-encoders. The results are evaluated using the USTC-TFC2016, CIC-IDS2017, as well as CSE-CIC-IDS2018 datasets. Table 1 describes the analysis of the existing work.

Problem statement: Although ML models are widely adopted in IDS, extensive research was needed to extract features from data produced by

Table 1
Existing works analysis.

Author Name & Reference	Technique used	Merits	Demerits	Dataset
Ons Aouedi et al. [21]	F-BIDS	Robust for secure and private applications.	Reduce scalability, increase complexity, and reduce efficiency.	Edge-IIoTset, InSDN dataset
Othmane Friha et al. [22]	FELIDS	Increase scalability, Data privacy, malicious activity detection.	Delay is high, costly	CSE-CIC-IDS2018, MQTTset and InSDN.
Md Mamunur Rashid et al. [23]	FL-ML	Increase accuracy to detect side-channel attacks.	Does not support all smartphone devices.	The edge-IIoTset dataset was used to evaluate the performance, and an accuracy of 92.49 % was achieved.
Yanu et al. [24]	FL	Attack recognition	Low accuracy	CIC-IDS-2017 dataset, as well as attained an accuracy of 97.1 %.
Idrissi et al. [25]	Fed-ANIDS	More accurate, Efficient	Cannot work in an unstable edge computing environment	USTC-TFC2016, CIC-IDS2017, as well as CSE-CIC-IDS2018

the huge volume of IoT devices. The DL model analyzes the data streams and extracts the necessary information. Consequently, the DL method was employed in multiple studies to identify anomalies, but the outcomes are inconclusive. Since most IDS are unsuitable for IoT applications because of issues with heterogeneity, resource limitations, privacy, etc. The most common IDS approach uses a centralized server or cloud to receive local data and use ML or DL to detect intrusions. The amount of time needed to classify a possible attack also becomes a barrier to stopping it from spreading throughout a network. Early attack detection is essential because generic malware has the ability to take control of susceptible IoT devices and spread quickly, creating botnets like Torri or Mirai. Thus, using cloud data centers was unaffordable, reducing the efficiency of intrusion detection. The IoT environment consisted of resource-constrained systems that communicated via wireless tools with restricted bandwidth and throughput. However, persistent device network connection incurs significant overhead in IoT networks with a large number of connected devices. To solve these issues, decentralized solutions are required, in which devices train independently without interacting with one another.

3. Proposed methodology

The usage of computer networks is growing rapidly in a range of functions, especially in vital infrastructure. The fast expansion of computer networks in today's vital infrastructure improves efficiency as well as reliability, but it also raises the risk of cyber-attacks. DL algorithms have lately been applied in a variety of applications; however, they frequently raise privacy concerns. FL has developed as a potential strategy for addressing these difficulties. Federated learning enables numerous systems or clients to train a model simultaneously without exchanging raw data. The workflow of the suggested FL-based intrusion detection model is shown in Fig. 1.

This study develops an **ensemble-based deep FL** for improving intrusion detection in a networking environment to detect threats

without data sharing. Initially, data pre-processing is performed, which includes **normalizing data and handling missing values**. The data cleaning method focuses on deleting null values as well as duplicate entries to eliminate bias in the most common data. This process is performed by converting data values to a proportionate range for every feature. This phase is crucial for preventing the dataset's bias towards characteristics with higher values. The suggested **ensemble-based deep FL** for an intrusion detection system consists of two parallel federated-based modules, namely, local and global modules, which combine the strength of ensemble methods and federated learning for enhanced intrusion detection. Each local module uses an ensemble of classifiers like **Residual block-based autoencoder, attention-based capsule network and stacked Bi-LSTM**, which are employed for the deep feature learning process, and the resultant output weight parameters are ensembled. Finally, the ensemble updates from each local module are combined in the FL server to yield an aggregated weight. The aggregated weight is then used to develop a new model in the global module that examines the network's normal and threat conditions. Thus, network attacks are identified without the sharing of data between users, thereby improving data security. The proposed method will be assessed against intrusion detection datasets.

3.1. Data pre-processing

The data collected has a large number of duplicated, unwanted, and missing values; therefore, the learning algorithms do not produce better results. As a result, data pre-processing is used to improve prediction accuracy by normalizing the data and removing noisy, irrelevant, and duplicate information. Data pre-processing includes data cleaning, normalization, and numerical transformation. The data provided as input is represented as follows,

$$D_n = \{D_1, D_2, \dots, D_s\} \quad (1)$$

where, D_n represents input data, s represents the number of information

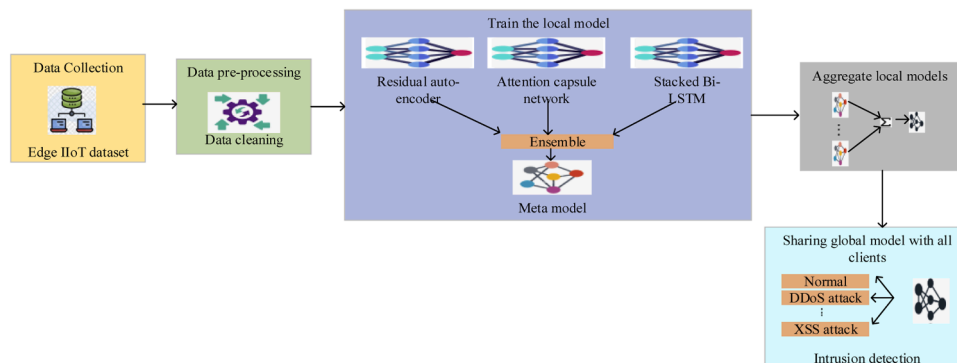


Fig. 1. Workflow of proposed FL-based intrusion detection model.

Data cleaning through handling missing values: The data cleaning procedure includes removing outliers, handling missing values, reducing noisy data, and replacing inconsistent data. However, inconsistent, noisy, and unwanted data are deleted, and missing data is handled by finding the mean or median of the available data. The missing data is filled by identifying the mean of the available data, which is reported as follows:

$$M_a = \frac{M_{a-1} + M_{a+1}}{2}, \quad a \in N \quad (2)$$

here, M_a signifies a missing value, M_{a-1} represents the preceding value of missed data, M_{a+1} represents the subsequent value of missed data, N represents usual numbers (i.e. 1, 2, 3, ...).

Data normalization: Data normalization is the procedure of scaling feature values into predetermined intervals. The unfairness in the raw data is eliminated, and every feature is positioned within a specific range based on the z-score, which is presented as follows:

$$Z(D_n) = \frac{X(D_n)_i + Y(D_n)_i}{\sigma X(D_n)_i}, \quad a \in N \quad (3)$$

where, $\sigma X(D_n)_i$ represents $X(D_n)_i$ data mean, $Y(D_n)_i$ denotes the standard deviation of the data. Thus, the pre-processed data is represented as follows,

$$(D_n)_i \xrightarrow{\text{pre-processing}} P(D_n)_i \quad (4)$$

where, $(D_n)_i$ represents data provided as input, $P(D_n)_i$ denotes pre-processed output.

3.2. Ensemble-based deep federated learning

The proposed ensemble-based deep federated learning system detects intrusions using anomalies. It includes global model initialization, local ensemble model training, model aggregation, as well as dissemination.

3.2.1. Federated learning system model

Federated learning has two main entities: the user and the server.

Federated learning is used to reduce communication costs and to secure users' privacy. The server is in charge of aggregating the data collected from multiple users. Thus, the proposed approach is designed to prevent collisions between users and servers. To avoid the loss of private information, the server only receives aggregated results from local users. All users do similar learning tasks with the same parameters and objectives. Each user evaluates their local weight from the training dataset and uploads it to the server. The server then computes the global weight by aggregating local weights and uploading them to the users. The attack is then detected using the newly generated model. The suggested federated learning process is revealed in Fig. 2.

Global model initialization: The central server began the learning procedure by establishing the global model's weights, which included the learning rate and epoch. The global model explored includes a residual block-based auto-encoder, an attention-based capsule network, and a stacked Bi-LSTM. Once the initialization process has begun, the central server sends the model as well as the hyperparameters to the local clients.

3.3. Local ensemble model training

Consider K number of clients to train the global model for intrusion recognition in the network. The clients train the ensemble method locally, earlier transfer the modified variables to the main server.

Learning process: The suggested malware detection system uses an ensemble-based deep FL model to ensure security. The suggested approach has two levels: local module and global module. The local module is taught on the client side, while the global module is trained on the server. The local module uses three classifiers, which are ensembles, to generate a meta-model. The three DL models, residual block-based auto-encoder, attention-based capsule network, and stacked Bi-LSTM, are used as base classifiers in the local module. They are trained on local data to generate model parameters, which are then ensemble to produce a meta-model. Each client transmits its meta-model parameters to the FL server for global aggregation. The FL server gathers meta-model weights from each client. At last, the server provides the collected model settings to each client. The proposed ensemble-based deep FL combines ensemble learning as well as FL. The suggested methodology uses FL to address security issues for the ensemble model

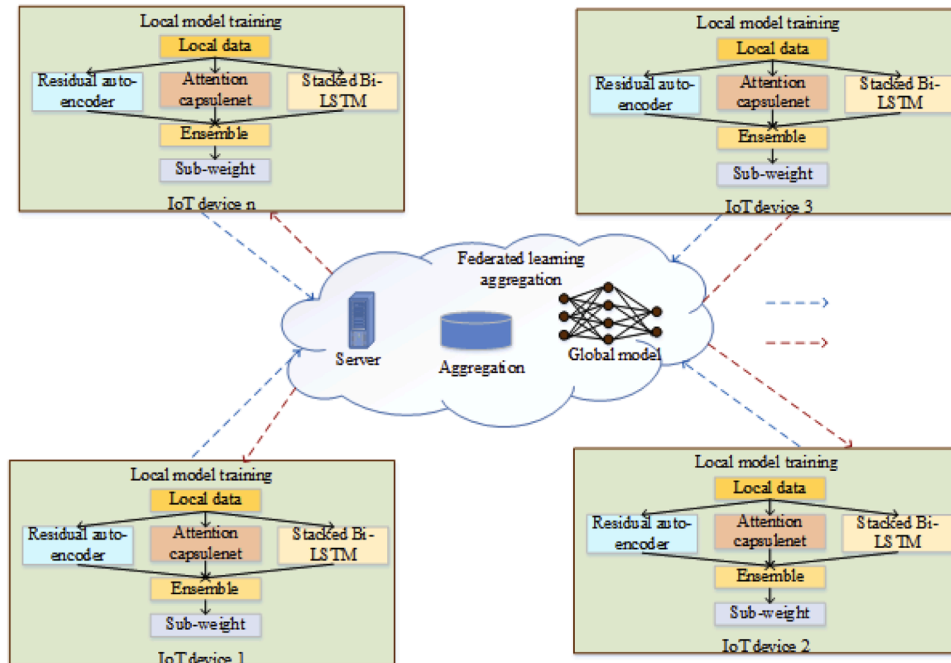


Fig. 2. Ensemble-based deep federated learning (E-DFL).

by providing parameter values rather than data. The ensemble model reduces the effectiveness of association implication attacks on the FL approach because, during each FL conversation round, the clients as well as the FL server transmit meta-model variables rather than base classifiers.

Residual Auto-encoder (AE): It is a type of neural network that produces a flattened demonstration of input data as an output [26]. The AE employs compressed data; therefore, it does not include any noise illustrations but simply includes the data's attributes. The architecture is shown in Fig. 3.

The AE consists of two units, namely, the encoder and decoder units. The encoder block decreases the dimensionality of the input data as well as maps it into the latent space, which is the hidden layer. While the decoder block learns the data representation from the latent space, it decodes incoming data. The AE is an unsupervised neural network designed for representation learning. The AE design includes a bottleneck layer that provides a flattened representation of the input. Thus, the flattened feature is erudite and then reconstructed at the output. The encoder moves the input data into the hidden layer, as well as the decoder rebuilds the original input. The encoding and decoding process is given using the following equation,

$$l = \delta(Va + y) \quad (5)$$

$$\hat{a} = \delta(Vl + y') \quad (6)$$

where, $a = (a_1, a_2, \dots, a_n)$ represents the input feature vector, $l = l_1, l_2, \dots$, l_n represents low dimensional vector obtained from the hidden layer, $\hat{a} = (\hat{a}_1, \hat{a}_2, \dots, \hat{a}_n)$ represents the reconstructed input, V, V' represents bias vectors, δ represents sigmoid activation function. The mean square error is considered as a reconstruction error between the l and $\hat{a}$ given as follows,

$$M = \frac{1}{n} \sum_{j=1}^n \frac{1}{2} \| a_j - \hat{a}_j \|^2 + \frac{\lambda}{2} (\| V \|^2 + \| V' \|^2) \quad (7)$$

where, λ represents the attenuation coefficient. The encoder unit compresses the input data; nevertheless, excessive compression affects reconstruction accuracy. Thus, the network's training stabilizes the denoising ability while maintaining modernization accuracy.

Residual block: When the network becomes deeper, information extracted from each stage vanishes while reaching the output layer. Deep networks have gradient vanishing problems, which slow down the training process. To address this issue, residual connections were used to send gathered features directly to the latter levels. It improves the flow of data as well as accelerates the training procedure. The residual block is given in Fig. 4.

Attention based Capsule network: CapsNets is a type of deep learning architecture designed to mitigate the issues associated with CNN architecture [27]. The capsule network consists of three layers:

convolutional, primary caps, and digit caps layers. The channel attention mechanism has been added to the capsule network. Capsules represent groups of neurons, while vectors represent parameters of specific activity. Capsule networks are strong in affine conversion and require less training data. The length of the network denotes the presence of an entity, whereas the orientation represents its qualities. The capsule network architecture is shown in Fig. 5.

Attention mechanism: DL methods rely heavily on attention mechanisms, particularly when dealing with sequential or spatial data. By establishing relationships between different channels or features, attention mechanisms recalibrate their importance and influence the classification results. If the weights of all features are reduced, discriminability will improve. The introduction of an attention mechanism focuses more on discriminative spatial and spectral aspects, weakening information that is negative for classification. The spectral as well as spatial attention modules, as shown in Fig. 6, were used to calibrate spectral and spatial features at several scales.

Channel attention module: This module is built by demonstrating the interdependencies between the channels, as exposed in Fig. 6. From the input feature map $F \in RS^{F \times h \times w}$, the spectral attention map $S \in RS^{F \times F}$ is evaluated where, $h \times w$ represents the spatial dimension, F represents the original features channel. Reshape and transpose the input features to get $S^T \in RS^{F \times n}$ then perform matrix multiplication between S and S^T the resultant output is then fed to a softmax layer to get the final attention map, given as follows,

$$Y_{ab} = \frac{\exp(F_a \times F_b)}{\sum_{a=1}^F \exp(F_a \times F_b)} \quad (8)$$

where, Y_{ab} represents the impact of a th channel in b th channel. Matrix multiplication is performed between F^T and S and the result is reshaped and transposed into $RS^{F \times h \times w}$. Finally, to normalize the weight, the scaling parameter is utilized, and an element-wise sum operation with the income to get the spectral attention map $P \in RS^{F \times h \times w}$.

$$P_a = \alpha \sum_{a=1}^F (Y_{ab} F_a) + F_a \quad (9)$$

The scale parameter is initially said to be zero, and the spectral attention feature map P is a weighted grouping of all the channels, thereby selecting the informative channels and suppressing the least useful ones.

Since the capsule network has multiple layers, the output of the capsule is evaluated using the non-linear squashing function given as follows,

$$N_q = \frac{\| X_q \|^2}{\varepsilon + \| X_q \|^2} \frac{X_q}{\| X_q \|} \quad (10)$$

where N_q represents the output vector of the capsule q , X_q represents the

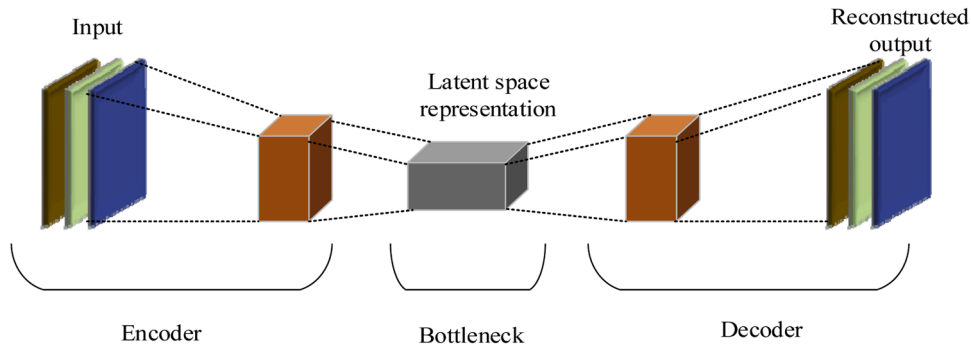


Fig. 3. Auto-encoder.

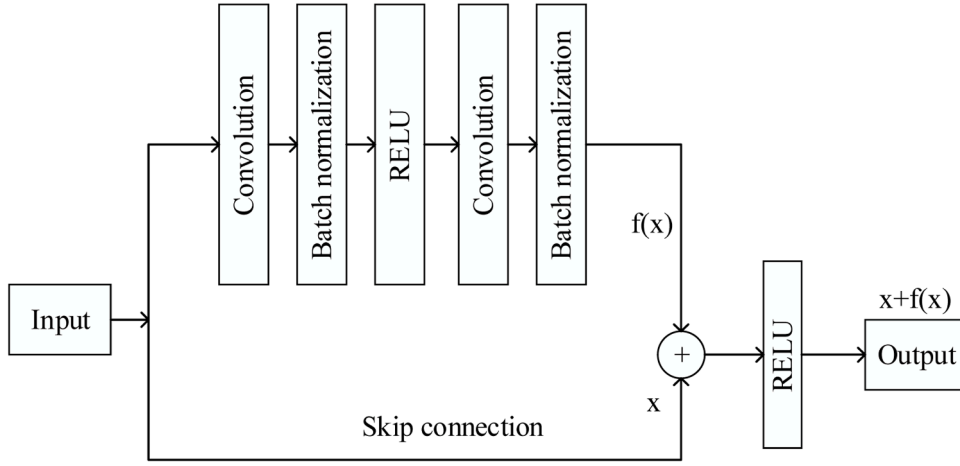


Fig. 4. Residual block.

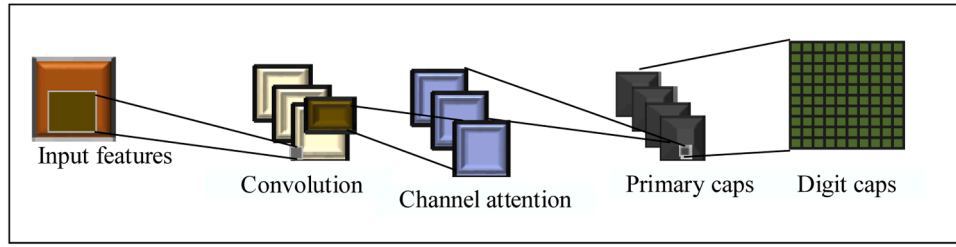


Fig. 5. Attention based Capsule network architecture.

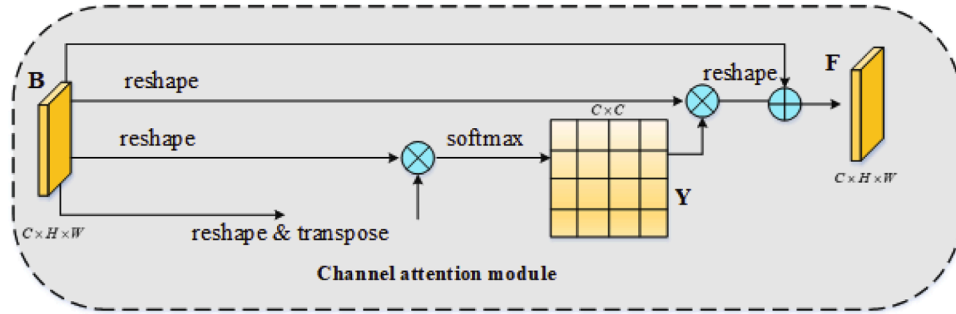


Fig. 6. Channel attention.

total input. The non-linear squash function is used as an activation function, which shrinks short vectors to length zero as well as long vectors to a particular length on the basis of ϵ .

$$X_q = \sum_q z_{pq} V_{pq} c_p \quad (11)$$

The input X_q is found by multiplying the results of the capsule with the weight matrix, z_{pq} represents the coupling coefficient evaluated by the dynamic routing process, given as follows,

$$z_{pq} = \frac{\exp(B_{pq})}{\sum_r \exp(B_{pr})} \quad (12)$$

where, B_{pq}, B_{pr} represents the log prior prospects among two capsules.

Each capsule layer sends outputs to the layer above, and the dissimilar transformation matrices and capsule types are combined to produce the same number of classes. During the iterative process, dynamic routing occurs, and lower-level capsules are forwarded to higher-level capsules. The overall length of the output vector denotes the

predicted probability.

3.3.1. Stacked Bi-LSTM

Stacked Bi-LSTM will improve classification performance over simpler models by capturing complex connections and long-range dependencies [28]. The classification performance of neural networks is further improved by stacking multiple BiLSTMs. In a stacked Bi-LSTM, output x_t from the bottom layer becomes the input of the upper layer in the stacked BiLSTM network. In Fig. 7, a layered BiLSTM structure is shown.

Every node of the hidden layer signifies an LSTM cell, which contains a memory, forget gate, input gate and output gate represented as U_t, IN_t, FR_t, OP_t . The input gate signifies new data stored in the cell. The forget gate represents the dumping of evidence from the cell, and the Output gate represents the output. The control parameter C_t represents which data to store or dump. The resultant output is evaluated using the following equations.

$$FR_t(a) = \delta(V_{FR}(a)y_t + W_{FR}(a)x_{t-1} + B_{FR}(a)) \quad (13)$$

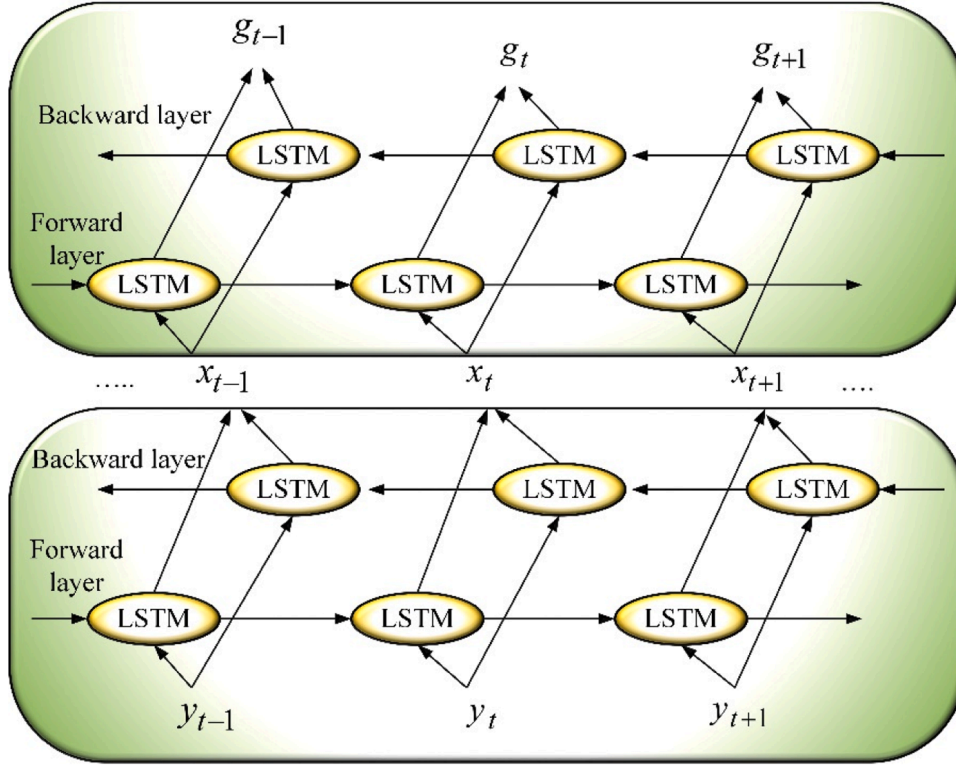


Fig. 7. Stacked Bi-LSTM.

$$IN_t(a) = \delta(V_{IN}(a)y_t + W_{IN}(a)x_{t-1} + B_{IN}(a)) \quad (14)$$

$$U_t(a) = \tanh(V_U(a)y_t + W_U(a)x_{t-1} + B_U(a)) \quad (15)$$

$$C_t(a) = FR_t(a) * C_{t-1}(a) + IN_t(a) * U_t(a) \quad (16)$$

$$OP_t(a) = \delta(V_{OP}(a)y_t + W_{OP}(a)x_{t-1} + B_{OP}(a)) \quad (17)$$

$$g_t = OP(a)_t * \tanh(C_t(a)) \quad (18)$$

Ensembling: The output obtained from individual models is integrated into a single output by fusing. Voting is the common fusion method used in classification problems to improve predictive performance. Here, the output from individual models is fused through the maximum voting method. The maximum voting is also known as hard voting or majority voting. The max voting method involves collecting predictions for every class label as well as forecasting one with the highest votes. The maximum voting is represented as follows,

$$M^* = \text{mod} [A_1(x), A_2(x), \dots, A_n(x)] \quad (19)$$

where, M^* represents the predicted class label through maximum voting of each classifier A_n .

$$M^* = \text{argmax} \sum_{i=1}^n v_i q_{ki} \quad (20)$$

where, v_i represents the weight assigned to i th classifier.

Model aggregation: This is the primary phase in the federated learning procedure. In a federated server, local ensemble models trained on each client are combined into a single global model. At the conclusion of every interaction round, every user spreads the ensemble model parameters to the server. The server then populates the worldwide model with the aggregated weights from the local ensemble models. Finally, the enhanced global model achieves intrusion detection tasks. The updates received from each IoT device are combined using the Federated

Averaging with Momentum (FedAvgM) approach. FedAvgM is a variant of federated averaging that includes driving into inform process. Old methods use momentum to speed up convergence. Similarly, FedAvgM uses historical gradients from contributing systems to rush the union of FL. FedAvgM is represented as follows:

$$\begin{aligned} W_t &= MO * W_{t-1} + L_R * G_t \\ U_t &= U_{t-1} - W_t \end{aligned} \quad (21)$$

where, W_t represents motion vector at time t , MO represents momentum coefficient, L_R represents learning rate, G_t represents average gradient, U_t represents the updated model.

$$MO \leftarrow \beta_{MO} + (1 - \beta)(\theta - \theta_{prev}) + Y \quad (22)$$

where, β represents the momentum parameter. The formula combined the previous value of MO that is the variance among current as well as previous global model parameters, as well as the aggregated local model update to compute the new value of MO . Eventually, a new global model with aggregated restrictions was developed, as well as attacks were detected using the updated global model. (Alg. 1)

4. Results and discussion

This part offers a detailed description of the experimental setup, dataset description, performance parameters used for result analysis, and simulation results from both proposed and existing studies. Initially, data gathered from the edge IIoT dataset is split in the ratio 80:20, with 80 % utilized for training as well as 20 % used for testing. The experimental setup utilized in this study is provided in Table 2.

4.1. Dataset description

This proposed work uses the Edge IIoT dataset, a cybersecurity dataset utilized in IoT as well as IIoT applications. The data in this dataset were collected using a range of sensors, such as heart rate

Algorithm 1

Federated learning process of the proposed ensemble-based deep federated learning model.

```

Input: Training edge IIoT dataset  $D = \{a_x, b_x\}$  ( $a_x \in A, b_x \in B$ )
Output: Intrusion detection using an ensemble algorithm
/*-train the base classifiers used
For  $y = 1$  to 2do in parallel
  Train  $M_y$  from  $D_{train}$ 
End for
/*- Construct metadata
 $D' = \{Q, B\}$  where  $Q = \text{concatenate } M_1, M_2, M_3 \text{ prediction}$ 
/*-Train the meta classifier in a federated way
For  $x = 1$  to  $R$ do
   $p = d_l * L$ 
  For  $e = 1$  to  $n$ do
    For  $f = 1$  to  $F$ do
      Train the Meta classifier parameters  $\alpha_e$  using  $D_e$ 
    End for
    Drive  $\alpha_e$  to the federated learning server
  End for
  Aggregate  $\alpha_e$  using federated averaging with momentum to get  $\alpha$ 
  Send the meta model to all clients
End for
Return  $R(A) = MC(M_1(A), M_2(A), M_3(A))$ 

```

Table 2

Experimentation setup.

Processor	Intel® Core(TM) i7-3245 CPU@3.40Ghz
Installed memory(RAM)	16.00 GB(15.83 GB usable)
System category	64-bit Operating system

sensors, flame sensors, temperature and humidity sensors, and so on. The dataset also comprised 14 various forms of assaults, including man-in-the-middle (MITM), DDoS, malware, injection, and so on. The dataset contains a total of 157,800 observations with 61 features and 15 possible labels; one is benign, and the other is a different type of attack. The data distribution is provided in Table 3.

4.2. Performance parameters measured

The performance parameters evaluated for the proposed and existing approaches are shown in Table 4. here, T_p , T_n denotes true positive and true negative rate, F_p , F_n denotes false positive as well as false negative rate.

4.3. Experimental evaluation

Performance of the proposed E-DFL approach is related to current federated learning-based approaches, including CNN, DNN, RNN, as well as CNN-BiLSTM [23,21], to evaluate its efficiency.

Table 3

Distribution of data in Edge IIoT dataset.

Type of attacks	Total sample
Normal	24,301
DDoS_UDP Attack	14,498
DDoS_ICMP Attack	14,090
Ransomware Attack	10,925
DDoS_HTTP Attack	10,561
SQL_injection Attack	10,311
Uploading Attack	10,269
DDoS_TCP Attack	10,247
Backdoor Attack	10,195
Vulnerability_scanner Attack	10,076
Port_Scanning Attack	10,071
XSS Attack	10,052
Password Attack	9989
MITM Attack	1214
Fingerprinting Attack	1001

Table 4

Metrics.

Parameters	Formula
Accuracy	$\frac{T_p + T_n}{T_p + T_n + F_p + F_n}$
Precision	$\frac{T_p}{T_p + F_p}$
Recall	$\frac{T_p}{T_p + F_n}$
Specificity	$\frac{T_n}{T_n + F_p}$
F1-score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

Fig. 8 demonstrates training as well as testing accuracy evaluated for suggested as well as current approaches using the Edge IIoT dataset. The epoch value varies from 0 to 300, and accuracy is evaluated during the training as well as the testing period. The suggested approach demonstrated higher accuracy while training as well as testing the data, demonstrating that the suggested approach works well.

Fig. 9 demonstrates the training as well as testing loss evaluated for proposed and existing methods using the edge IIoT dataset. The epoch value is varied from 0 to 300, and the loss value is calculated during training as well as testing periods. The proposed approach has been validated by demonstrating a lower loss value in both training as well as testing the data.

Fig. 10 depicts the performance of the FL-based centralized DL method in regard to accuracy, specificity, F1-score, recall and precision. The investigation demonstrates how successfully the model distinguishes benign as well as attack classes in the edge IIoT dataset. The suggested approach obtained an accuracy of 99.23 %, specificity of 97.24 %, recall of 98.6 %, precision of 99 %, f-score of 99 %, and a time of 4.5 min, whereas the CNN produced fewer results than the other approaches. The suggested method outperformed previous methods in recognizing benign and malicious assaults. Despite their effectiveness, centralized methods are prone to single point-of-failure difficulties. However, the outcomes indicate that the suggested E-DFL technique outperformed the current methods. The values achieved are provided in Table 5.

Fig. 11 shows the accuracy measurement of proposed and existing methods for a varied distribution of data by varying the FL rounds from 0 to 50. However, the proposed method has achieved higher accuracy than the current approaches, as shown in Table 6.

Fig. 12 displays the ROC curve of the Edge-IIoT dataset. The ROC

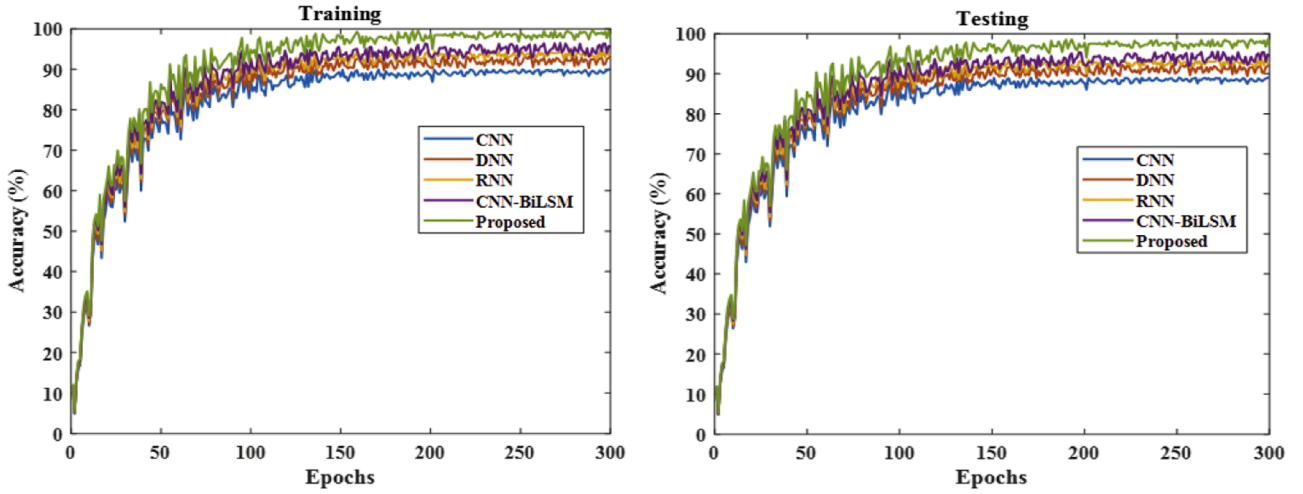


Fig. 8. Evaluation of training and testing accuracy.

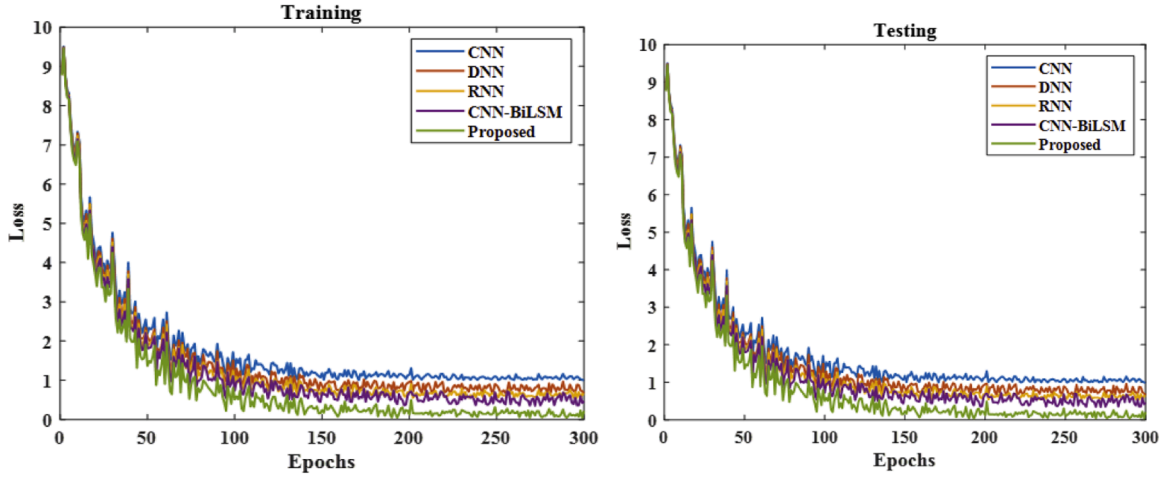


Fig. 9. Evaluation of training and testing loss.

curve plots the true positive rate and false positive rate of intrusion detection. The TPR is also said to be the detection Rate, as well as FPR is also said to be the False Alarm Rate. Compared to the AUC values of the existing method, the suggested FL technique obtained better results. The proposed approach obtained an AUC of 99.23 %, whereas the other CNN model achieved the lowest of 92 %, DNN achieved 96.5 %, RNN achieved 96.2 %, and CNN-Bi-LSTM achieved 98.5 %, respectively.

Fig. 13 displays the training time of the proposed FL method for different numbers of clients, such as 3, 9, and 15, for IID and non-IID data distribution. The term "IID data" refers to data distribution that is consistent with each client's data distribution throughout the dataset. The term "non-IID data" describes a data distribution that is not consistent with the delivery of data for every user within the dataset. In comparison with existing methods, the proposed method achieved improved accuracy values for IID and non-IID data. Compared to the RNN method, the CNN method requires more training time. The data distribution has no impact on the time consumption. The training time of the suggested as well as current approaches depends on the number of clients used, as exposed in Table 7.

To ensure rigorous comparison, all experiments were executed over $n = 10$ independent runs, each with different random seeds and shuffled train-test splits while preserving the 80:20 ratio. For each metric, the mean and standard deviation across these 10 runs are provided. Paired statistical tests were performed by computing for every run the difference between the proposed E-DFL model and each baseline on the same

split, thus satisfying the requirement of paired sampling. Specifically, a paired t -test was applied to evaluate whether the mean performance difference was statistically at $\alpha = 0.05$. However, because accuracy based measures can violate normality assumptions, which additionally include non-parametric significance tests for robustness. A McNemar test was applied to paired confusion matrices to compare classification decisions on identical test samples, and bootstrap confidence intervals (95 % CI) were generated using 10,000 resamples to quantify the variability of accuracy, precision, recall and F1-score without relying on distributional assumptions. These complementary non-parametric analyses reinforce that the observed improvements of the E-DFL model over existing FL-based IDS methods are consistent and statistically reliable.

Fig. 14 depicts the communication overhead analysis for proposed as well as current technologies. The suggested approach had a lower communication overhead than the other current methods.

Fig. 15 shows a confusion matrix analysis of the suggested approach using the edge-IIoT dataset. The dataset has 15 types of attacks. The proposed method correctly detected 51 normal attacks, 48 DDoS-UDP attacks, 41 fingerprint attacks, 59 MITM attacks and 56 XSS attacks. As a result, more attacks are correctly detected, demonstrating the proposed method's increased performance. The proposed model is empirically evaluated to assess the privacy robustness.

4.3.1. Empirical evaluation of privacy protection

The empirical evaluation was conducted by targeting two major

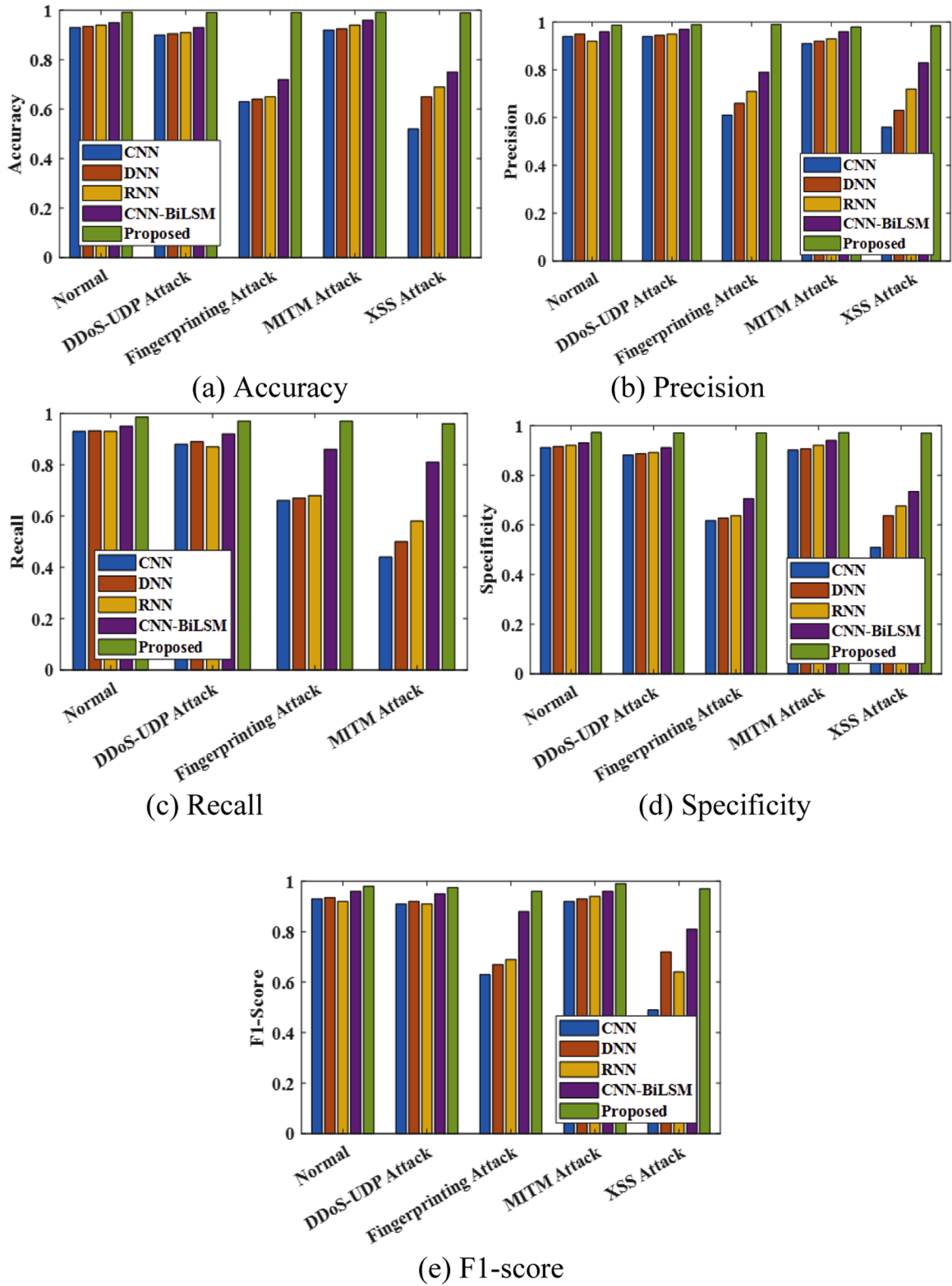


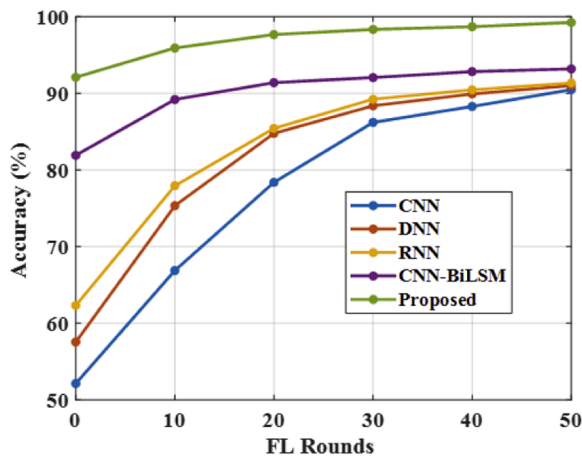
Fig. 10. Analysis of Performance. (a) Accuracy (b) Precision (c) Recall (d) Specificity. (e) 1-score.

threat models commonly associated with federated learning using membership inference attacks (MIA) and model inversion attacks (MIA-Inv). These evaluations were performed using simulated adversarial clients that attempted to extract information from the shared model updates without accessing raw data.

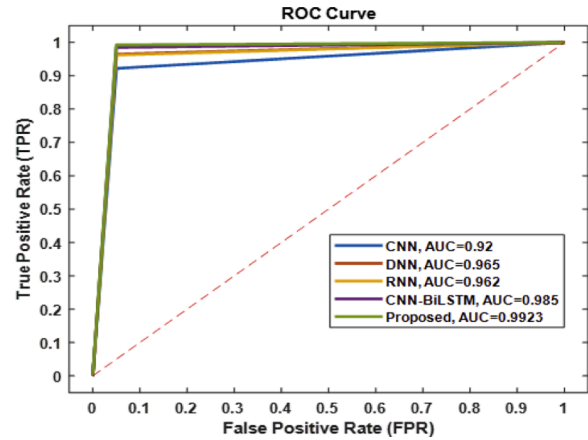
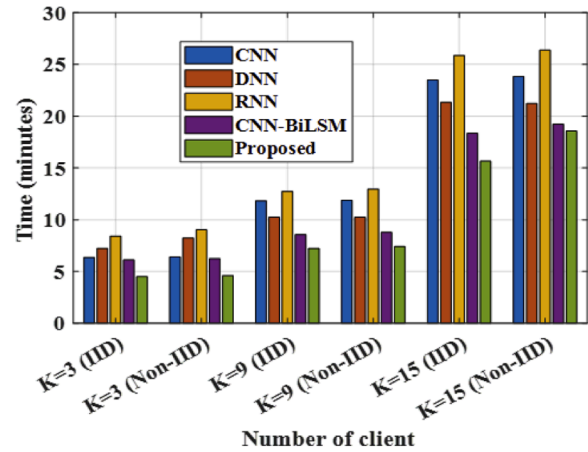
4.3.1.1. Membership inference attack (MIA) evaluation. A shadow method based inference attack was implemented in which a challenger efforts to define whether an exact sample was amount of a client's local training dataset. The attack classifier was trained using the gradients exchanged during FL rounds, confidence scores of the global model and loss distribution of member vs. non-member samples. Across 10 runs, the attacker achieved an average inference accuracy of 51.8 % which is

Table 5
Performance analysis.

Accuracy					
Attacks/method	CNN	DNN	RNN	CNN-BiLSM	Proposed method
Normal	0.93	0.935	0.94	0.95	0.9923
DDoS-UDP attack	0.9	0.905	0.91	0.93	0.991
Fingerprint attack	0.63	0.64	0.65	0.72	0.991
MITM attack	0.92	0.925	0.94	0.96	0.992
XSS attack	0.52	0.65	0.69	0.75	0.99
Precision	CNN	DNN	RNN	CNN-BiLSM	Proposed method
Normal	0.94	0.95	0.92	0.96	0.987
DDoS-UDP attack	0.94	0.945	0.95	0.97	0.989
Fingerprint attack	0.61	0.66	0.71	0.79	0.99
MITM attack	0.91	0.92	0.93	0.96	0.98
XSS attack	0.56	0.63	0.72	0.83	0.985
Recall	CNN	DNN	RNN	CNN-BiLSM	Proposed method
Normal	0.93	0.932	0.93	0.95	0.986
DDoS-UDP attack	0.88	0.89	0.87	0.92	0.97
Fingerprint attack	0.66	0.67	0.68	0.86	0.97
MITM attack	0.44	0.5	0.58	0.81	0.96
XSS attack	0.44	0.55	0.58	0.87	0.976
F-score	CNN	DNN	RNN	CNN-BiLSM	Proposed method
Normal	0.93	0.935	0.92	0.96	0.98
DDoS-UDP attack	0.91	0.92	0.91	0.95	0.975
Fingerprint attack	0.63	0.67	0.69	0.88	0.96
MITM attack	0.92	0.93	0.94	0.96	0.99
XSS attack	0.49	0.72	0.64	0.81	0.97
Specificity	CNN	DNN	RNN	CNN-BiLSM	Proposed method
Normal	0.9114	0.9163	0.9212	0.931	0.972454
DDoS-UDP attack	0.882	0.8869	0.8918	0.9114	0.97118
Fingerprint attack	0.6174	0.6272	0.637	0.7056	0.97118
MITM attack	0.9016	0.9065	0.9212	0.9408	0.97216
XSS attack	0.5096	0.637	0.6762	0.735	0.9702

**Fig. 11.** Accuracy of Intrusion Detection using multiple clients with a varied distribution of data.**Table 6**
Accuracy Vs FL rounds.

Accuracy					
FL Rounds	CNN	DNN	RNN	CNN-BiLSM	Proposed method
0	52.13	57.55	62.31	81.91	92.09
10	66.88	75.33	77.93	89.19	95.91
20	78.38	84.77	85.42	91.38	97.66
30	86.2	88.37	89.23	92.05	98.33
40	88.28	89.91	90.45	92.83	98.68
50	90.47	91.01	91.34	93.18	99.24

**Fig. 12.** RoC curve analysis.**Fig. 13.** Training time.**Table 7**
Training time.

Time (min)					
No. of clients/method	CNN	DNN	RNN	CNN-BiLSM	Proposed
K = 3 (IID)	6.35	7.23	8.41	6.12	4.5
K = 3 (Non-IID)	6.41	8.23	9.02	6.23	4.6
K = 9 (IID)	11.82	10.23	12.73	8.56	7.21
K = 9 (Non-IID)	11.87	10.23	12.95	8.78	7.4
K = 15 (IID)	23.49	21.34	25.86	18.34	15.65
K = 15 (Non-IID)	23.83	21.22	26.37	19.23	18.56

close to random guessing (50 %). This indicates that the E-DFL design, particularly the exchange of only meta model parameters instead of full gradient, substantially reduces membership information leakage.

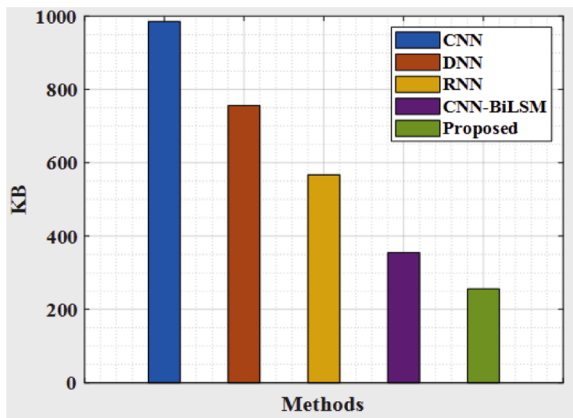


Fig. 14. Communication overhead analysis.

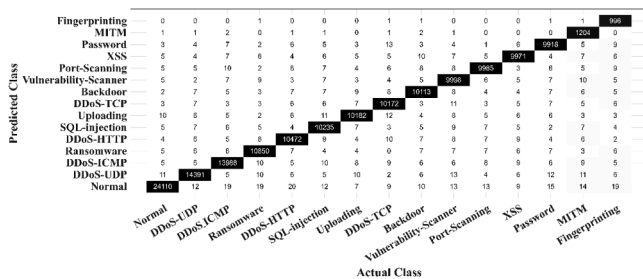


Fig. 15. Confusion matrix.

4.3.1.2. Model inversion attack (MIA-Inv) evaluation. An evaluation is conducted to determine whether an adversarial client could reconstruct input features from the aggregate global model. A gradient prior based inversion technique was applied to recover input approximations for three randomly selected clients. The reconstructed samples exhibited extremely low structural similarity ($SSIM < 0.08$) and bore no identifiable resemblance to the original feature vectors. This demonstrates that the multi-level ensemble architecture and the absence of raw gradient sharing significantly hinder inversion capabilities. The empirical results ensure that the E-DFL framework prevents attackers from reliably distinguishing member from non-member samples. Reconstructing meaningful feature information from global updates is not feasible under the proposed architecture. The privacy leakage risk is considerably lower compared with traditional FL setups that share full model gradients. The empirical privacy evaluation validates that the proposed E-DFL approach offers strong practical privacy protection by limiting shared information and incorporating ensemble based aggregation.

4.4. Discussion

The efficacy of the suggested work was associated with existing FL-based IDS techniques. The comparisons are based on datasets, ML classifiers, client count, and data distribution methods. The proposed method handled both IID and non-IID data concerns while also demonstrating performance for both data types. The performances are assessed using the edge IIoT dataset. The proposed E-DFL approach is considered effective for the following reasons. The usage of FL provides a more secure as well as consistent transmission of data in IoT devices. The hidden IoT data is not accessed from the server because the server only stores aggregated model parameters, ensuring data privacy and security. IoT devices can detect intrusions even when they are disconnected. The results reveal that as the number of federated learning rounds increased, the proposed technique maintained similar accuracy. This is because, after each cycle, the model's performance was increased

by learning from all client-ends, and it performed as exactly as a centralized model. Table 8 shows a comparison of the suggested study with current studies.

5. Conclusion and future scope

The research focused on using E-DFL to secure IoT data. The increasing amount of cyber-attacks in networked systems makes it harder to develop a scalable intrusion detection system. Thus, the proposed E-DFL model combines the residual autoencoder, stacked Bi-LSTM, and attention capsule net to create an aggregate global model for detecting intrusions in the edge IIoT dataset. The data is trained locally for each user, and just the local training parameters are exchanged for collaborative learning, ensuring that the original data remains secure from potential assaults. The E-DFL model successfully detected intrusions, and the results prove that the suggested approach obtained competitive results in both IID and non-IID scenarios. In the future, an FL model using Raspberry Pi will be created to explore the potential of FL in resource-constrained environments. Also, the presence of dishonest clients and servers may raise privacy concerns. Thus, more investigation is required into the topic of creating a more trustworthy FL by eliminating all potential risks.

Funding information

No funding is provided for the preparation of the manuscript.

CRedit authorship contribution statement

Dhanraj Somaling Jadhav: Writing – original draft, Visualization, Supervision, Conceptualization. **Sneha R Jadhav-Mane:** Writing – review & editing, Validation, Software. **Kayan Devappa Bamane:** Supervision, Project administration, Conceptualization. **Sonal Suresh Mohite:** Visualization, Supervision, Software. **Abhijit Janardan Patankar:** Writing – review & editing, Formal analysis, Conceptualization. **Arati Sidu Gaikwad:** Supervision, Software, Resources.

Declaration of competing interest

No conflict of Interest.

Table 8
Comparison of existing studies.

Author Name & Reference	Technique used	Dataset	Performances evaluated
Ons Aouedi et al. [21]	F-BIDS	Edge-IIoTset,	Precision-97 %, recall-99 %, f-score-98 %
		InSDN dataset	Precision-99 %, recall-99 %, f-score-98 %
Othmane Friha et al. [22]	FELIDS	CSE-CIC-IDS2018	Accuracy-94.22 %
		MQTTset	Accuracy-90.76 %
Md Mamunur Rashid et al. [23]	FL-ML	InSDN	Accuracy-98.54 %
		Edge-IIoTset dataset	accuracy of 92.49 %
Yanu et al. [24]	FL	CIC-IDS-2017 dataset and achieved	Accuracy of 97.1 %.
Idrissi et al. [25]	Fed-ANIDS	USTC-TFC2016	Accuracy-83.72 %, f-score-80.53 %, FDR-7.98 %
		CIC-IDS2017	Accuracy-86.76 %, f-score-58.18 %, FDR-0.14 %
		CSE-CIC-IDS2018	Accuracy-89.95 %, f-score-85.44 %, FDR-2.41 %
Proposed method	E-DFL	Edge IIoT dataset	Accuracy- 99.23 %, precision-99 %, recall- 98.6 %, specificity- 97.24 %, f-score-99 %, time- 4.5min

Data availability

The authors do not have permission to share data.

References

- [1] S. Lee, M. Mohammadi, S. Rashidi, A.M. Rahmani, M. Masdari, M. Hosseinzadeh, Towards secure intrusion detection systems using deep learning techniques: comprehensive analysis and review, *J. Netw. Comput. Appl.* 187 (2021) 103111, <https://doi.org/10.1016/j.jnca.2021.103111>.
- [2] J. Asharf, M.N.H. Khurshid, E. Debie, W. Haider, A. Wahab, A review of intrusion detection systems using machine and deep learning in internet of things: challenges, solutions and future directions, *Electronics* 9 (7) (2020) 1177, <https://doi.org/10.3390/electronics9071177>.
- [3] J.A. Abraham, V.R. Bindu, Intrusion detection and prevention in networks using machine learning and deep learning approaches: a review, in: 2021 International Conference on Advancements in Electrical, Electronics, Communication, Computing and Automation (ICAECA), IEEE, 2021, pp. 1–4.
- [4] Z. Ahmad, A.S. Khan, C.W. Shiang, J. Abdullah, F. Ahmad, Network intrusion detection system: a systematic study of machine learning and deep learning approaches, *Trans. Emerg. Telecommun. Technol.* 32 (1) (2021) e4150, <https://doi.org/10.1002/ett.4150>.
- [5] J. Lansky, S. Ali, M. Mohammadi, M.K. Majeed, S.H.T. Karim, S. Rashidi, M. Hosseinzadeh, A.M. Rahmani, Deep learning-based intrusion detection systems: a systematic review, *IEEE Access* 9 (2021) 101574–101599, <https://doi.org/10.1109/ACCESS.2021.3097247>.
- [6] U.S. Musa, M. Chhabra, A. Ali, M. Kaur, Intrusion detection system using machine learning techniques: a review, in: 2020 International Conference on Smart Electronics and Communication (ICOSEC), IEEE, 2020, pp. 149–155, <https://doi.org/10.1109/ICOSEC49089.2020.9215333>.
- [7] S. Gamage, J. Samarabandu, Deep learning methods in network intrusion detection: a survey and an objective comparison, *J. Netw. Comput. Appl.* 169 (2020) 102767, <https://doi.org/10.1016/j.jnca.2020.102767>.
- [8] M.A. Alsoufi, S. Razak, M.M. Siraj, I. Nafea, F.A. Ghaleb, F. Saeed, M. Nasser, Anomaly-based intrusion detection systems in iot using deep learning: a systematic literature review, *Appl. Sci.* 11 (18) (2021) 8383, <https://doi.org/10.3390/app11188383>.
- [9] L. Ashiku, C. Dagli, Network intrusion detection system using deep learning, *Procedia Comput. Sci.* 185 (2021) 239–247, <https://doi.org/10.1016/j.procs.2021.05.025>.
- [10] Y. Otoum, D. Liu, A. Nayak, DL-IDS: a deep learning-based intrusion detection framework for securing IoT, *Trans. Emerg. Telecommun. Technol.* 33 (3) (2022) e3803, <https://doi.org/10.1002/ett.3803>.
- [11] I. Idrissi, M. Azizi, O. Moussaoui, IoT security with Deep Learning-based Intrusion Detection Systems: a systematic literature review, in: 2020 Fourth International Conference on Intelligent Computing in Data Sciences (ICDS), IEEE, 2020, pp. 1–10, <https://doi.org/10.1109/ICDS50568.2020.9268713>.
- [12] K. Roshan, A. Zafar, Deep learning approaches for anomaly and intrusion detection in computer network: a review, in: *Cyber Security and Digital Forensics: Proceedings of ICCSDF 2021*, 2022, pp. 551–563.
- [13] S.V. Amanoul, A.M. Abdulazeez, Intrusion detection system based on machine learning algorithms: a review. 2022 IEEE 18th International Colloquium On Signal Processing & Applications (CSPA), IEEE, 2022, pp. 79–84, <https://doi.org/10.1109/CSPA55076.2022.9782043>.
- [14] D.N.P. Suthishni, K.S.S. Kumar, A review on machine learning based security approaches in intrusion detection system, in: 2022 9th International Conference on Computing for Sustainable Global Development (INDIACom), IEEE, 2022, pp. 341–348, <https://doi.org/10.23919/INDIACom54597.2022.9763261>.
- [15] O.H. Abdulganiyu, T.A. Tchakoucht, Y.K. Saheed, A systematic literature review for network intrusion detection system (IDS), *Int. J. Inf. Secur.* 22 (5) (2023) 1125–1162, <https://doi.org/10.1007/s10207-023-00682-2>.
- [16] A. Halbouni, T.S. Gunawan, M.H. Habaebi, M. Halbouni, M. Kartiwi, R. Ahmad, Machine learning and deep learning approaches for cybersecurity: a review, *IEEE Access* 10 (2022) 19572–19585, <https://doi.org/10.1109/ACCESS.2022.3151248>.
- [17] N. Unnisa, M. Yerva, M.Z. Kurian, Review on intrusion detection system (ids) for network security using machine learning algorithms, *Int. Res. J. Adv. Sci. Hub* 4 (3) (2022) 67–74, <https://doi.org/10.47392/irjash.2022.014>.
- [18] S. Tsimenidis, T. Lagkas, K. Rantos, Deep learning in IoT intrusion detection, *J. Netw. Syst. Manag.* 30 (1) (2022) 8, <https://doi.org/10.1007/s10922-021-09621-9>.
- [19] E.M. Campos, P.F. Saura, A.G. Vidal, J.L.H. Ramos, J.B. Bernabe, G. Baldini, A. Skarmeta, Evaluating federated Learning for intrusion detection in Internet of Things: review and challenges, *Comput. Netw.* 203 (2022) 108661, <https://doi.org/10.1016/j.comnet.2021.108661>.
- [20] A. Belenguer, J. Navaridas, and J.A. Pascual. A review of federated learning in intrusion detection systems for iot. *arXiv preprint arXiv:2204.12443* (2022).
- [21] A. Ons, K. Piamrat, F-bids: federated-blending based intrusion detection system, *Pervasive Mob. Comput.* 89 (2023) 101750.
- [22] O. Friha, M.A. Ferrag, L. Shu, L. Maglaras, K.K.R. Choo, M. Nafaa, FELIDS: federated learning-based intrusion detection system for agricultural Internet of Things, *J. Parallel. Distrib. Comput.* 165 (2022) 17–31, <https://doi.org/10.1016/j.jpdc.2022.03.003>.
- [23] M.M. Rashid, S.U. Khan, F. Eusufzai, M.A. Redwan, S.R. Sabuj, M. Elsharief, A federated learning-based approach for improving intrusion detection in industrial internet of things networks, *Network* 3 (1) (2023) 158–179, <https://doi.org/10.3390/network3010008>.
- [24] R. Yang, H. He, Y. Wang, Y. Qu, W. Zhang, Dependable federated learning for IoT intrusion detection against poisoning attacks, *Comput. Secur.* 132 (2023) 103381, <https://doi.org/10.1016/j.cose.2023.103381>.
- [25] M.J. Idrissi, H. Alami, A.E. Mahdaoui, A.E. Mekki, S. Oualil, Z. Yartaoui, I. Berrada, Fed-anids: federated learning for anomaly-based network intrusion detection systems, *Expert. Syst. Appl.* 234 (2023) 121000, <https://doi.org/10.1016/j.eswa.2023.121000>.
- [26] V. Ciric, A. Milojkovic, M. Milosevic, Autoencoder-based network intrusion detection on multiple datasets, in: 2024 IEEE 22nd Mediterranean Electrotechnical Conference (MELECON), IEEE, 2024, pp. 1281–1286, <https://doi.org/10.1109/MELECON56669.2024.10608696>.
- [27] Z. Yang, Q. Huang, Q. Wang, X. Zong, R. Ao, Visual intrusion detection based on CBAM-Capsule networks, *Comput. J.* (2024), <https://doi.org/10.1093/comjnl/bxae011>.
- [28] A. Bhattarai, U. Gyawali, V. Abhishek, V. Ranga, Improving intrusion detection in a software defined network using hybrid CNN and Bi-LSTM, in: 2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAIC), IEEE, 2024, pp. 117–122, <https://doi.org/10.1109/ICAIC60222.2024.10575090>.